Ingeniería de Software

Qué es el Software:

- Conjunto de programas de cómputo, procedimientos, reglas, documentación y datos asociados que forman parte de un sistema informático.
- **Incluye:**
    1. **Instrucciones (programas)** que ejecutan funciones y características.
    2. **Estructuras de datos** para manipular información.
    3. **Información descriptiva** (manuales, documentación, ayuda en línea).
- A diferencia del hardware, no se manufactura, sino que se desarrolla o modifica intelectualmente.
- El software no se desgasta, aunque puede quedar obsoleto.

Importancia del Software en la Vida Cotidiana:

- **Presente en todo:** banca, salud, actividades hogareñas, educación, industria.
- Desde objetos simples (ej. tostadora) hasta sistemas críticos (ej. hospitales).
- Nuestra vida depende cada vez más de sistemas informáticos → crece la importancia de producir software confiable y de calidad.

Ingeniería de Software: Es la disciplina que aplica principios de ingeniería al desarrollo de software, buscando calidad, confiabilidad y eficiencia en costos.

Bauer(1972): Ingeniería del Software trata del establecimiento de los principios y métodos de la ingeniería a fin de obtener software de modo rentable, que sea fiable y trabaje en máquinas reales.

Bohem(1976): Ingeniería del Software es la aplicación práctica del conocimiento científico en el diseño y construcción de programas de computadoras y la documentación asociada requerida para desarrollar, operar y mantenerlos. Se conoce también como desarrollo de software o producción de software.

Zelkovitz(1978): Ingeniería de Software es el estudio de los principios y metodologías para el desarrollo y mantenimiento de sistemas de software.

IEEE(1993): La aplicación de un enfoque sistemático, disciplinado y cuantificable al desarrollo, operación (funcionamiento) y mantenimiento del software; es decir, la aplicación de ingeniería al software.

Método: Los métodos de la Ingeniería de Software indican cómo construir software. Incluyen una gran gama de tareas como el análisis de requisitos, diseño, elaboración de programas, pruebas y mantenimiento.

Herramientas: Las herramientas de la Ingeniería de Software indican proporcionan un soporte para el proceso y para los métodos. Pueden ser automáticas y semiautomáticas.
Son instrumentos que permiten realizar algo con mayor exactitud o de mejor forma. Nos ayudan a ser más productivos o eficientes o refuerzan la calidad del producto resultante.

Actividad: como el conjunto de tareas o acciones realizadas para lograr un objetivo.

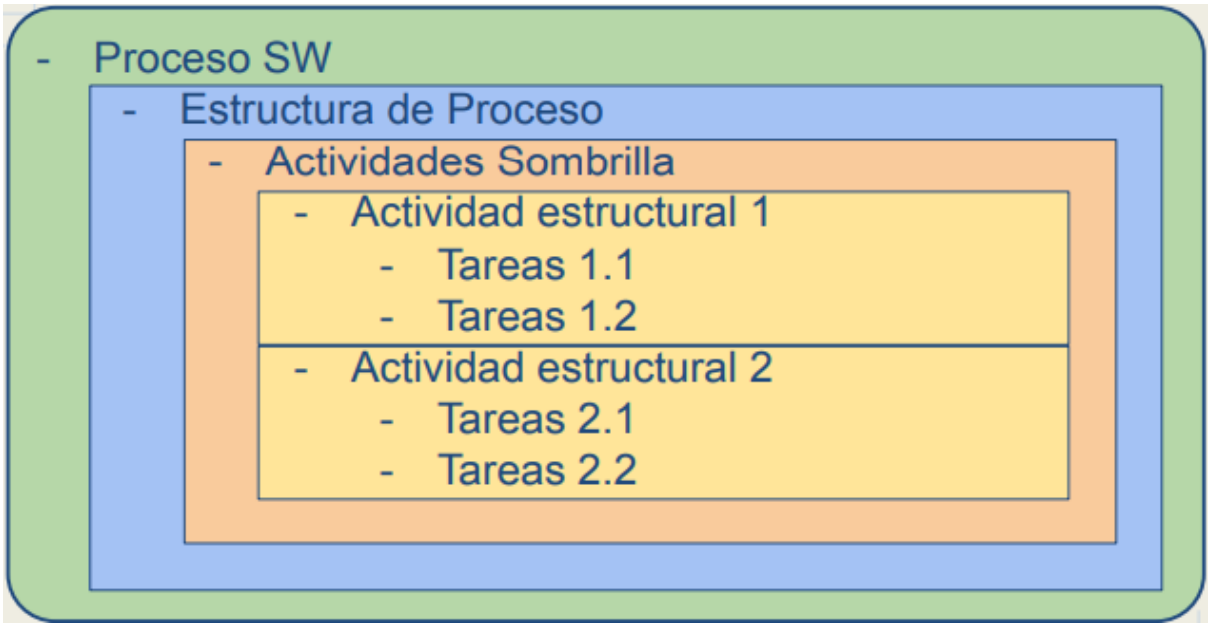
Proceso: Proceso en Ingeniería de Software define el marco de trabajo para un conjunto de las actividades del desarrollo del software y lo abarca en totalidad.

Paradigma: Es un *enfoque o filosofía* para la construcción de software (ejemplo: desarrollo orientado a objetos).

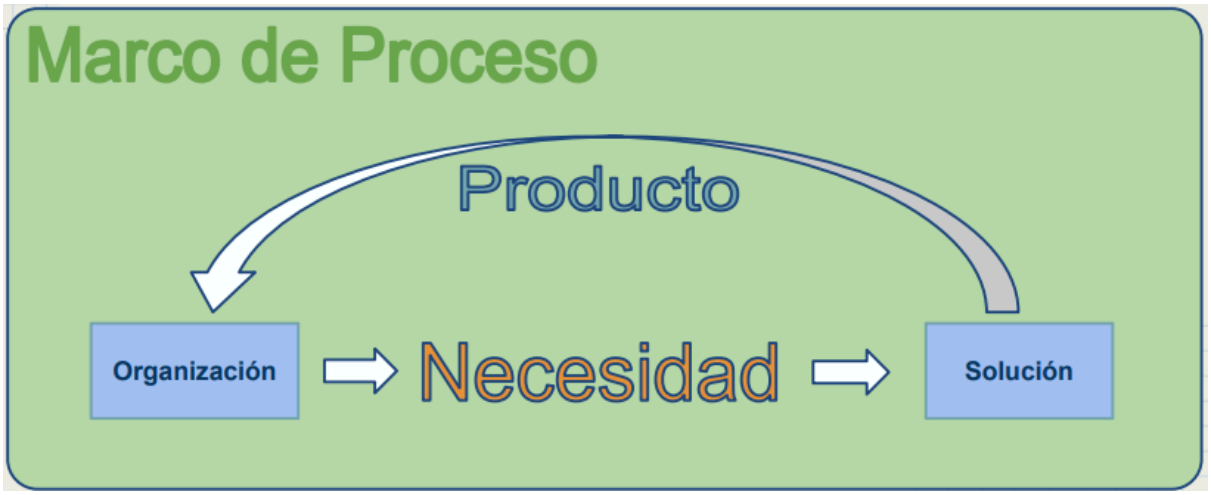
Aclarar estos conceptos ayuda a dar un marco más claro al desarrollo de la disciplina y a mejorar la comunicación entre profesionales, clientes y usuarios.

Informática y Ciencias de la Computación: A nivel internacional, *Computer Science* se usa como sinónimo de Informática. Este término abarca tanto el software como el hardware (computadora como objeto de estudio).
El ingeniero de software usa la computadora como una *herramienta* dentro de la solución, pero no es su objeto principal de estudio.

Proceso Software

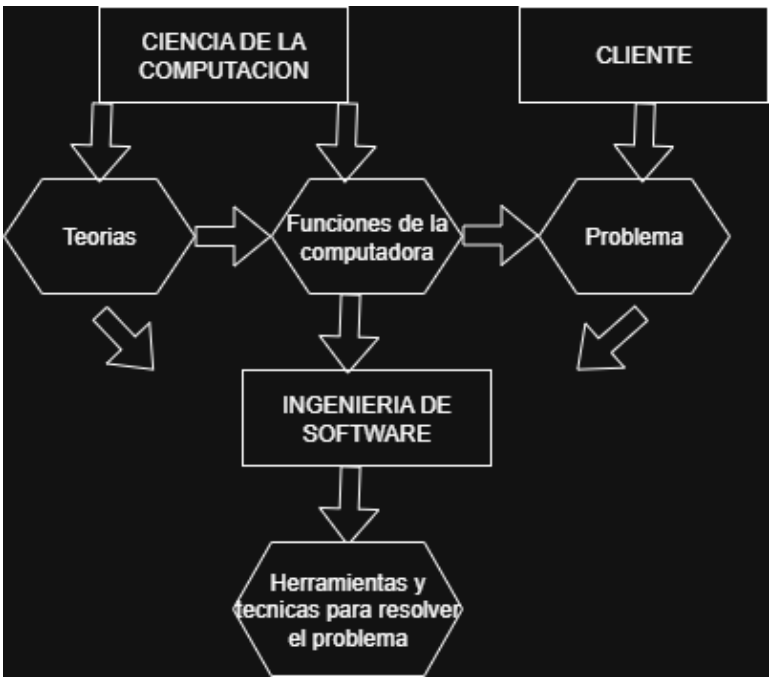


Producto - Proceso



Pfleeger

Ubicación de la Ingeniería de Software (Pfleeger): Pfleeger mostró en un gráfico cómo la Ingeniería de Software se ubica dentro de la Ciencia de la Computación. De la computación provienen las teorías y las funciones de la computadora, y del cliente surgen los problemas. La Ingeniería de Software toma todo eso y aplica herramientas y técnicas para dar soluciones.
Esta visión no es compartida por todos, pero ayuda a entender la relación entre disciplinas.



Rol de la Ingeniería de Software:La Ingeniería de Software se considera una disciplina de diseño y desarrollo de software de alta calidad.
Los ingenieros de software usan herramientas, técnicas, procedimientos y paradigmas para obtener soluciones eficientes y productivas.
La improvisación es el peor camino: si una empresa trabaja sin método ni planificación, no puede garantizar seriedad ni continuidad en el mercado.

Calidad en el Software:El tema de la calidad es central pero también complejo:

- Hay distintas visiones sobre qué significa “calidad” y cómo medirla.
- En todo caso, la calidad debe estar presente en todo el ciclo de vida del software, no solo en la codificación.
- Se busca que el software sea confiable, seguro, planificado a largo plazo y capaz de resistir malos usos.

Muchas veces la calidad se ve limitada por las exigencias del mercado, que obligan a lanzar productos rápido, sin pruebas suficientes. Por eso es común que las primeras versiones tengan errores y que se vayan corrigiendo en versiones posteriores. Otra causa frecuente de fallas es la falta de preparación en los equipos de prueba, que suelen enfocarse solo en las funciones más usadas.

Costos de los Errores:Corregir un error temprano es mucho más barato que hacerlo tarde.

Ejemplo: Un error en la fase de análisis cuesta aproximadamente una décima parte de lo que costaría corregirlo después de entregado el software.
Por eso, asegurar la calidad desde el principio ahorra costos y evita problemas graves.

Técnicas y Normas de Calidad:Existen técnicas muy simples y útiles como las revisiones y las inspecciones, que permiten detectar 4 de cada 5 errores durante la construcción.

La IEEE ha desarrollado normas internacionales específicas para apoyar el aseguramiento de la calidad en software.

La calidad debe hacerse desde tres perspectivas (ver como poder resumir esto

1. Producto
2. Proceso
3. Ámbito del negocio o dominio

1. Producto: La calidad que se ve y se usa.
    - Usuario: Le importa que funcione bien, sea fácil de usar y cumpla su propósito.
    - Desarrollador/Mantenimiento: Les importa la calidad interna (código bien estructurado, fácil de modificar y corregir).
2. Proceso: La calidad en *cómo* se hace.
    - Un proceso de desarrollo bien organizado y con controles (usando normas como CMMI, ISO 9000) asegura y garantiza un producto final de calidad.
3. Ámbito del Negocio (Dominio):
    - El software debe cumplir con los objetivos y necesidades específicas del negocio o área para la que fue creado (ej: banca, salud, retail).

Conclusión clave: No basta con que el producto *parezca* bueno para el usuario. La calidad real se logra cuando se combina un buen proceso para crear un producto técnicamente sólido que satisfaga las necesidades del negocio.

Cambios en la Ingeniería de Software

Las tradicionales actividades del desarrollo del software: Análisis y definición de requisitos, Diseño del sistema, Diseño de programas, Prueba unitaria, Prueba de integración, Prueba de Sistema, Entrega del Sistema y mantenimiento, en un modo ideal, parecen sugerir una secuencia lógica, que en la realidad no

siempre es posible. En muchos casos las actividades se repiten para ajustarlas, por lo que en el proceso de desarrollo software estas nueve actividades se organizan de diferentes formas para lograr un mejor resultado, dadas las características de un determinado proyecto. Los proyectos alcanzan mayor tamaño día a día, crece la complejidad en cuanto a la interacción entre diferentes tipos de software, protocolos de comunicación, redes de gran alcance, sistemas embebidos, etc. Todos estos vertiginosos cambios han provocado también cambios en la Ingeniería de Software que busca adaptarse como disciplina a las necesidades del desarrollo de software.

Wasserman, en su libro “Towards a discipline of software engineering”, analiza los cambios en la Ingeniería de Software, definiendo las causas en 7 factores que los originan:

1. Criticidad del tiempo de puesta en el mercado para los productos comerciales.
2. Variaciones en los factores económicos de la computación, menores costos del hardware y mayores costos del desarrollo y el mantenimiento.
3. Disponibilidad de la poderosa computación de escritorio.
4. Uso intensivo de redes locales y de área amplia.
5. Disponibilidad y adopción de tecnología orientada a objetos.
6. Interfaces gráficas de usuario, usando ventanas, iconos y punteros.
7. La falta de predicción del modelo de desarrollo de software en cascada.

Otro aporte de Wasserman son las ocho nociones fundamentales que forman las bases para una disciplina efectiva de la Ingeniería de Software. Ellas son:

1. Abstracción: Permite realizar una descripción del problema desde un nivel de generalización que permita concentrarse en los conceptos fundamentales. La abstracción permite realizar diferentes niveles de conceptualización del problema.

2. Métodos y notaciones de análisis y diseño: No existe en Ingeniería de Software una notación que permita la comunicación entre pares. Esta situación conlleva malas interpretaciones constituyéndose en unos de los problemas claves de la Ingeniería de Software. Los métodos de análisis y diseño, aportan además de un medio de comunicación, la posibilidad de construir modelos y probarlos en cuanto a completitud y consistencia.

3. Prototipado de la interfaz de usuario: Esta técnica permite construir una versión reducida de un sistema, una funcionalidad limitada, que puede usarse para ayudar al usuario o al cliente a identificar los requerimientos del sistema, o para demostrar la factibilidad de un diseño o enfoque determinado. También

puede favorecer aspectos o suposiciones no aclarados en la reducción de requisitos.

4. Arquitectura del software: La arquitectura es una noción sumamente importante en todo el proceso software ya que afectan no sólo a la facilidad de implementación, prueba, velocidad, efectividad de mantenimiento y el cambio. La calidad de la arquitectura es definitiva en un sistema. Algunos autores, sostienen que la arquitectura puede constituirse como una disciplina en sí misma, que persigue como fin último la división en módulos arquitectónicos lo más independientes posible de otros.

5. El proceso del software: La organización y la disciplina en las actividades han sido reconocidas por contribuir a la calidad del software resultante. Sin embargo, diferentes productos necesitan de diferentes procesos y no existe una receta que cubra todas las necesidades de todos los desarrollos.

6. Reutilización: Permite sacar ventaja de los aspectos comunes de las aplicaciones, reutilizando elementos de desarrollos previos. Se pueden reutilizar conjuntos de requerimientos, partes de diseños o grupos de guiones de prueba. Si bien la reutilización es un objetivo claro para las empresas de desarrollo de software, no es fácil su implementación debido a muchos factores como tiempo de desarrollo de módulos reutilizables, difícil documentación, difícil de establecer el grado de calidad del módulo con la calidad exigida para un proyecto, etc.

7. Medidas: A través de las diferentes áreas de investigación que intentan mejorar la Ingeniería de Software un aspecto clave para la mejora ha sido la medición, la que permite establecer dónde se está y hacia dónde se pretende ir, constituyendo una de las “buenas prácticas” de la Ingeniería de Software, ya que permite comparar, evaluar y retroalimentar el proceso software.

8. Herramientas y ambientes integrados: Este factor hace referencia a la utilización de herramientas CASE las que favorecen un ambiente integrado y estandarizado que mejora el proceso de desarrollo, si bien, la confluencia de dos características del desarrollo hacen difícil esta integración: por un lado los problemas son diferentes y requieren de diferentes enfoques en el proceso software, por lo que el uso de diferentes procesos, métodos y re-Ingeniería de Software - Ingeniería de Software -Definiciones Definiciones Definiciones Mst. Nora Gigante 7 cursos hacen difícil la unificación del enfoque. Sin embargo, la existencia de herramientas cada vez mejores, permiten analizar y concluir cuál es la más apropiada para un problema o una aplicación.

El Proceso Software y los Modelos de Ciclo de Vida

Elegir un Ciclo de vida, permite encadenar las tareas y actividades del Proceso Software. Una vez conseguido un proceso software concreto para un proyecto determinado se está preparado para planificar los plazos del proyecto, asignar personas a las diferentes tareas, presupuestar costos del proyecto, etc.

Los procesos software, hoy día se usan como:

- Guía descriptiva sobre la documentación a producir y enviar al cliente
- Bases para determinar qué herramientas, técnicas y metodologías de Ingeniería de Software serán más apropiadas para brindar soporte a las diferentes actividades.
- Marco para analizar o estimar patrones de asignación y consumo de recursos a lo largo de todo el ciclo de vida del producto software.
- Base para llevar estudios empíricos para determinar qué afecta a la productividad, costo y calidad del software.
- Descripción comparativa sobre cómo los sistemas llegan a ser lo que son.

El proceso software está compuesto de cuatro procesos principales:

1. Proceso de selección de un modelo de ciclo de vida
2. Proceso de gestión del proyecto
3. Procesos orientados al desarrollo del software
4. Procesos integrales del proyecto

Estos procesos abarcarán todo el ciclo de vida del producto desde su comienzo hasta su retiro por obsolescencia o por reemplazo, incluyendo dentro del mismo, el mantenimiento.

Modelos de Ciclos de Vida

El Ciclo de Vida del Proyecto (o Modelo de Ciclo de Vida, MCV) es simplemente la secuencia de fases por las que pasa un proyecto desde su inicio hasta su finalización.
En esencia, responde a la pregunta: "¿En qué orden vamos a hacer el trabajo y cuándo vamos a obtener *feedback* sobre el producto?"

Hay 3 categorías principales:

1. Lineal/Cascada
2. Prototipado

3. Evolutivo

MCV Cascada o Lineal: El modelo más antiguo. Propone un flujo secuencial estricto, como una línea de manufactura.

Ventajas (Por qué se usaba)	Desventajas (Por qué se necesita Scrum)
Organización Lógica: Estructura de fases coherente, simple y fácil de entender.	**Alto Riesgo y Coste de Error:** Los errores o malentendidos se detectan en las fases finales (Pruebas/Mantenimiento), cuando son extremadamente costosos de corregir.
Revisión y Control: Cada fase incluye una revisión formal para minimizar errores antes de avanzar.	**Falta de flexibilidad:** Su naturaleza secuencial estricta (no permite volver atrás) dificulta la gestión de cambios, lo que es inevitable en proyecto complejos.
Simplicidad: Proporciona una visión clara y simple del proceso de desarrollo, ideal para proyectos con requisitos fijos.	**Definición Temprana Imposible:** Exige obtener todos los requisitos al inicio, algo irreal, ya que el cliente los descubre o cambia con el tiempo.

Fases del Modelo en Cascada

Este modelo se divide en las siguientes etapas principales:

1. **Ingeniería y modelado de sistemas:** Se establecen los requisitos del sistema completo, incluyendo el hardware, y se asigna el subconjunto de requisitos correspondientes al software.
2. **Análisis de requisitos:** Se profundiza en los requisitos específicos del software para entender el dominio de la información, su función, comportamiento y rendimiento. Los requisitos se documentan y son revisados por el cliente.
3. **Diseño:** Se traducen los requisitos a una representación evaluable del software, que incluye la estructura de datos, la arquitectura, las interfaces y los algoritmos.
4. **Codificación:** Se genera el código del software basándose en el diseño detallado.
5. **Pruebas de integración:** Se verifica que el software cumpla con los requisitos definidos y que la funcionalidad sea correcta.

6. **Mantenimiento:** El software se adapta y mejora después de la entrega al cliente, ya sea para corregir errores, ajustarse a cambios en el entorno o agregar nuevas funcionalidades.

Modelo prototipado: El Modelo de Prototipado es una técnica de desarrollo adaptativa que busca superar las limitaciones del Cascada. Su objetivo principal es aclarar requisitos y reducir el riesgo creando una versión preliminar del sistema (prototipo) para que el cliente interactúe con él en etapas tempranas para ayudar a comprender los requisitos del usuario. Este modelo busca superar las limitaciones del modelo en cascada, al permitir que el cliente interactúe con una versión del sistema en las primeras etapas.

Etapas del Proceso:

1. **Análisis preliminar:** Recolectar requisitos iniciales (aunque sean incompletos).
2. **Desarrollo del prototipo:** Crear la versión funcional básica.
3. **Prueba del prototipo:** El cliente usa el prototipo y proporciona retroalimentación.
4. **Refinamiento:** Se ajusta el prototipo o se actualizan las especificaciones de requisitos, repitiendo el ciclo.
5. **Sistema final:** Una vez que los requisitos son claros, se construye el sistema definitivo.

Ventajas	Desventajas (Riesgos a Considerar)
Reducción de Riesgo: Permite validar requisitos y diseños en etapas tempranas.	**Riesgo de Enfoque:** Si el equipo se enfoca demasiado en refinar un prototipo Desechable, puede volverse caro y difícil de descartar.
Mejor Entendimiento: El cliente interactúa con el prototipo y aclara sus verdaderas necesidades.	**Riesgo de "Producto Final Rápido":** Los clientes pueden malinterpretar el prototipo como el producto final con errores y funcionalidad limitada.
Supera la Cascada: Aborda la limitación del Cascada al obtener feedback temprano antes de invertir grandes recursos en el desarrollo completo.	**Documentación Incompleta:** La rapidez del prototipado puede llevar a una menor formalidad o detalle en la documentación final.
Naturaleza Iterativa: Facilita la adaptación y mejora el producto basándose en el uso real.	

Tipos de Prototipos

Tipo de Prototipo	Objetivo y Destino	Uso Clave
Desechable	Explorar y Aclarar requisitos. El código se descarta una vez que se aprende lo necesario.	Se enfoca en aspectos malentendidos o desconocidos.
Maqueta (Mockup)	Representación visual (interfaz de usuario) con datos estáticos (no reales).	Útil para dar un ejemplo visual al usuario sin construir la lógica.
Evolutivo	Se construye y se refina continuamente. Se convierte en el sistema final (no se tira).	Ideal para construir a partir de requisitos claros y conocidos, añadiendo funcionalidad gradualmente.

El modelo de prototipado es muy útil para la validación de requisitos y diseños, ya que permite al cliente ver y usar una versión del producto antes de que se inviertan grandes cantidades de recursos en el desarrollo completo. Su naturaleza iterativa ayuda a reducir el riesgo de que el producto final no cumpla con las expectativas del usuario.
comprender los requisitos del usuario. Este modelo busca superar las limitaciones del modelo en cascada, al permitir que el cliente interactúe con una versión del sistema en las primeras etapas.

Modelo Evolutivo: Los modelos evolutivos son enfoques de desarrollo de software que a diferencia del modelo en cascada, reconocen que los requisitos cambian con el tiempo. Estos modelos construyen el sistema de forma incremental o iterativa, permitiendo que el software evolucione con el tiempo.

Característica Clave	Explicación
Adaptación al Cambio:	Los requisitos cambian conforme avanza el desarrollo, y el modelo es altamente flexible para absorberlos.
Entrega Temprana:	Se crea un producto sin todos los requisitos para salir lo antes posible (entrega rápida de valor).
Incertidumbre Aceptada:	Se sabe que hay incertidumbre sobre la cantidad de ciclos necesarios y la velocidad de evolución.
Producto Esencial:	Se empieza con las funcionalidades básicas o fundamentales y se trabaja con detalles inciertos.

1.Modelo Incremental: El modelo incremental (también llamado de emisión gradual) combina aspectos del modelo en cascada con la flexibilidad del prototipado. El sistema se divide en subsistemas funcionales, y el desarrollo se realiza en versiones o incrementos.

Enfoque	Ventajas	Desventajas
Construcción: Se divide el sistema en subsistemas. Cada ciclo AÑADE NUEVA funcionalidad completa.	**Entrega Rápida de Valor:** El cliente recibe un producto funcional parcial desde el principio.	**Riesgo de Arquitectura:** El producto esencial inicial debe tener una arquitectura lo suficientemente buena para soportar futuras adiciones.
Ejemplo: V1 (Crear texto), V2 (Añadir cortar/pegar), V3 (Añadir formato).	**Gestión de Riesgos:** Posibilidad de posponer componentes inciertos.	
Recursos: Permite iniciar con poco personal e ir escalando en futuras entregas.		

2. Modelo Iterativo: El modelo iterativo (o de refinamiento sucesivo) también se basa en ciclos repetitivos, pero su enfoque es diferente al incremental.

Enfoque	Ventajas	Desventajas
Construcción: Se entrega un sistema completo desde el principio, pero con funcionalidad primitiva.	**Feedback Temprano:** Permite recibir *feedback* del cliente sobre todas las funciones (aunque sean simples).	**Riesgo de Priorización:** Si no se prioriza bien, se puede caer en un ciclo interminable de refinamiento sin nuevas características.
Mejora: Cada ciclo MEJORA y REFINA la funcionalidad EXISTENTE (calidad, eficiencia, usabilidad).	**Calidad Mejorada:** El refinamiento continuo basado en el uso real mejora la calidad final del producto.	
Ejemplo: V1 (Funciones lentas y simples), V2 (Mismas funciones, pero más rápidas y eficientes).		

3. Modelo Espiral: El modelo espiral es un enfoque avanzado y dirigido por los riesgos. Combina la naturaleza iterativa con la gestión del riesgo de forma explícita.

Enfoque	Ventajas
Proceso: Combina la naturaleza iterativa con la gestión explícita del riesgo.	**Ideal para Proyectos Grandes/Complejos:** Su orientación al riesgo es su mayor fortaleza.
Ciclo: Cada vuelta de espiral incluye: Planificación → Análisis de Riesgos → Ingeniería → Evaluación.	**Mitigación Proactiva:** Identifica y resuelve riesgos (técnicos, presupuestarios) antes de que afecten gravemente el proyecto.
Flexibilidad: Puede incorporar elementos de Cascada o Prototipado según el riesgo a tratar.	**Reutilización:** Se presta bien a la reutilización de *software* y a proyectos a largo plazo (no distingue entre desarrollo y

	mantenimiento).

En la práctica, muchos proyectos combinan estos modelos, utilizando un enfoque iterativo e incremental para entregar nuevas funcionalidades y, al mismo tiempo, mejorar las ya existentes. Esto permite un desarrollo más ágil y adaptable a las necesidades cambiantes del cliente.

Gestión de Proyectos

¿Qué es Administrar?

Ordenar, disponer, organizar, en especial la hacienda o los bienes.

Gobernar: Ejercer la autoridad o el mando sobre un territorio y sobre las personas que lo habitan.

Suministrar: Proporcionar o distribuir algo.

Graduar o dosificar el uso de algo: para obtener mayor rendimiento de ello o para que produzca mejor efecto.

¿Quién? Que? Como?

Todos
Algo: Personal, Laboral.

- Ej el tiempo, las finanzas, las actividades
- En general son recursos.

En nuestro caso debemos comprender las 4 P:

- Personal
- Producto
- Proceso
- Proyecto

Concepto: “Administrar es un concepto que suele referirse al acto de graduar la utilización de algo para obtener un rendimiento mejor, se trate de una sustancia, una acción o simplemente del tiempo”.

Conceptos Clave de la Gestión de Proyectos de Software:

La gestión de proyectos de software es fundamental para evitar el fracaso, que lamentablemente aún es muy común en la industria. Muchos proyectos fallan debido a una gestión débil, que a menudo se manifiesta en problemas técnicos y de planificación.

La gestión eficaz de un proyecto se enfoca en las "4 P 's": Personal, Producto, Proceso y Proyecto.

Personal

Desde 1960 Se estudia la formación de personal Motivado y Capacitado
El factor humano es tan importante que se desarrolló el People-CMM.
El factor humano es el más importante. El éxito de un proyecto depende de contar con un equipo de desarrollo preparado, motivado y con talento. La gestión del personal incluye actividades como el reclutamiento, la capacitación, la evaluación del rendimiento y la retención del talento.

Modelo de madurez del personal:

- Mejorar habilidades
- Motivar, desarrollar, mejorar y conservar la fuerza de trabajo
- **Define áreas claves como:**
    - ☐ Ambiente de trabajo / Clima
    - ☐ Capacitación
    - ☐ Competencias
    - ☐ Desarrollo profesional
    - ☐ Bonificación por desempeño
    - ☐ Beneficios

El Rol Crucial del Personal en el Desarrollo de Software:

El factor humano es, sin duda, el elemento más importante para el éxito de un proyecto de software. Como señalan varios vicepresidentes de tecnología, la gente y el talento del equipo son más valiosos que las herramientas o el proceso. Sin embargo, a pesar de reconocer su importancia, a menudo se da por sentado.

Los Participantes:

- **Gestores superiores:** Definen los objetivos de negocio y tienen una gran influencia en el proyecto.
- **Gestores de proyecto:** Planifican, organizan, motivan y controlan al equipo técnico.

- **Profesionales:** Son los ingenieros y técnicos que realizan el trabajo de desarrollo.
- **Clientes:** Especifican los requisitos y necesidades del software.
- **Usuarios finales:** Son quienes usarán el software una vez que esté terminado.

Los líderes equipo: Un buen líder de equipo es fundamental, pero no siempre el mejor técnico es el mejor gestor. Según el modelo MOI de Jerry Weinberg, un gestor eficaz debe tener tres cualidades clave:

- **Motivación:** La habilidad para inspirar al equipo a rendir al máximo.
- **Organización:** La capacidad de estructurar el proceso para transformar una idea en un producto.
- **Ideas (Innovación):** La capacidad de fomentar la creatividad dentro del equipo.

Además, un buen gestor de proyectos debe ser un resolutor de problemas y tener la confianza para tomar el control cuando sea necesario, al tiempo que permite la autonomía de su equipo. También debe recompensar los logros y fomentar un espíritu de equipo fuerte.

Estructuras del Equipo de Software:

La forma en que se organiza un equipo impacta su productividad y moral. Existen tres estructuras de equipo comunes:

- **Descentralizado democrático (DD):** Sin un líder fijo, las decisiones se toman por consenso. Es ideal para resolver problemas complejos y para equipos que estarán juntos por mucho tiempo.
- **Descentralizado controlado (DC):** Con un líder y sublíderes, la toma de decisiones es colaborativa, pero la implementación es asignada.
- **Centralizado controlado (CC):** El líder toma las decisiones de alto nivel y la comunicación es vertical. Es más eficiente para tareas sencillas.

Un equipo eficaz debe ser cohesivo, o "cuajar", lo que significa que sus miembros confían entre sí y comparten un objetivo común. Los equipos "tóxicos" (con alta frustración, roles confusos o fallos repetidos) deben ser evitados, y los gestores deben enfocarse en eliminar estas "toxinas" y en fomentar la colaboración.

Los líderes equipo:

- Requiere mucho trato con la gente
- Habilidades blandas
- Modelo MOI para liderazgo técnico

- Motivación
- Organización
- Ideas/Innovación Los lideres equipo
- Gerentes de proyecto
- Resolución de problemas
- Identidad ○ Incentivos por logro
- Influencia y construcción de equipo

El equipo SW:

Cada equipo tiene una estructura y depende de:

- Organización
- Dificultad del problema
- Tamaño del producto
- Tiempo
- Grado de modularización
- Rigidez de la fecha
- Grado de sociabilidad/comunicación

Producto

- Se debe pensar en el producto antes de pensar o planear un proyecto
- Objetivos y Ámbito
- Analizar alternativas
- Restricciones
- Sino es muy difícil hacer estimaciones precisas
- Costos
- Riesgos
- Tiempos
- ¿Se puede descomponer?

Antes de iniciar cualquier planificación, es crucial definir claramente los objetivos y el ámbito del producto. Esto implica entender lo que el cliente necesita, las funciones que debe tener el software y los posibles desafíos técnicos. Sin esta claridad, es imposible hacer estimaciones de costos o planificaciones realistas.

El Producto en la Gestión del Proyecto: Al comienzo de un proyecto, el gestor necesita un plan y estimaciones, pero a menudo los requisitos del software son inciertos o cambian. La primera actividad es definir el ámbito del software, respondiendo a tres preguntas clave:

- **Contexto:** ¿Cómo encaja el software en un sistema más grande y qué limitaciones existen?
- **Objetivos de información:** ¿Qué datos de entrada y salida manejará?
- **Función y rendimiento:** ¿Qué funciones realizará y qué características de rendimiento son necesarias?

El ámbito debe ser claro, sin ambigüedades y delimitado con datos cuantitativos (ej: número de usuarios simultáneos). Si una característica no puede delimitarse, se considera un riesgo principal del proyecto.

Descomposición del Problema:Para crear un plan, se aplica la estrategia de "divide y vencerás", descomponiendo el problema en partes más pequeñas y manejables.

Esto se hace en dos áreas:

1. **Descomposición de la funcionalidad:** Se desglosan las funciones del software en subfunciones más detalladas (ej: "procesador de textos" se divide en "comprobación ortográfica", "revisión gramatical", etc.).
2. **Descomposición del proceso:** Se eligen las tareas de ingeniería de software que conforman el modelo de proceso. La Estructura Común de Proceso (ECP), que incluye actividades como comunicación, planificación e ingeniería, sirve como base invariable para cualquier proyecto. Sin embargo, las tareas específicas dentro de cada actividad varían según el proyecto. Por ejemplo, la actividad de "comunicación con el cliente" para un proyecto simple tendrá menos tareas que para un proyecto complejo.

Proceso

- Proporciona el marco conceptual
- Se debe elegir el marco “adecuado” que pueda adaptarse
    - Personal, producto y entorno
- Variar la cantidad de actividades
    - Se deben seleccionar los procesos/subprocesos correctos
- Actividades sombrilla o “cross”
    - Ocurren a lo largo del proceso
    - No importa el tamaño del SW ni el marco de proceso elegido

El proceso software proporciona la estructura sobre la cual se construye el proyecto. Se trata de un conjunto de actividades que se adaptan a las características del proyecto y a los requisitos del equipo. Un proceso bien definido, junto con actividades de apoyo como la gestión de la calidad, asegura que el desarrollo sea eficiente y de alta calidad.

1. Marco Adaptable:

- El proceso proporciona un marco de trabajo, pero no es rígido.
- Se debe elegir y adaptar el marco (metodología) según el personal, el producto y el entorno del proyecto.

2. Actividades Fundamentales (ECP): Existe un núcleo de actividades estructurales que son invariables y se aplican a TODOS los proyectos, sin importar su tamaño o metodología. Estas son:

- Comunicación con el cliente (Obtención de requisitos)
- Planificación (Recursos y tiempos)
- Análisis de riesgo (Evaluar riesgos)
- Ingeniería (Diseño y representación del software)
- Construcción y entrega (Programar, probar, instalar)
- Evaluación del cliente (Feedback)

3. Descomposición y Flexibilidad:

- Las actividades siempre son las mismas, pero las tareas específicas para cumplirlas SÍ varían.
- Un proyecto pequeño y simple requerirá pocas tareas rápidas para cada actividad.
- Un proyecto grande y complejo requerirá muchas más tareas detalladas y formales para la misma actividad.

Conclusión clave: Se debe aplicar siempre el mismo conjunto de actividades fundamentales (ECP), pero la cantidad y profundidad de las tareas se adaptan flexiblemente a las necesidades específicas de cada proyecto.

Proyecto

- **Se planean y se controlan:**
    - Única forma de controlar su complejidad
- Se de debe definir cómo Planificar, Monitorear y Controlar
    - Gerente proyecto es el responsable
- Se deben establecer indicadores que notifiquen cierto parámetros
    - Alertas
    - Semáforos

Dirigir un proyecto planificado y controlado es la única forma de manejar la complejidad inherente al desarrollo de software. Un gestor debe identificar y evitar las señales de peligro y aplicar un enfoque de sentido común para planificar, supervisar y controlar el proyecto. Este enfoque es crucial para entregar un producto de alta calidad a tiempo y dentro del presupuesto.

El Proyecto(Cómo Evitar el Fracaso):Un proyecto de software puede fallar por muchas razones. John Reel identificó diez señales de peligro que indican que un proyecto está en problemas, como:

- Falta de entendimiento de las necesidades del cliente.
- Ámbito del producto mal definido o en constante cambio.
- Fechas de entrega poco realistas.
- Personal sin las habilidades adecuadas o con alta rotación.
- Evitar las buenas prácticas de gestión.

Un proyecto que no se gestiona bien puede caer en la regla del 90-90: el 90% del esfuerzo se dedica a los primeros 90% del proyecto, mientras que el 10% restante del proyecto consume el otro 90% del tiempo y el esfuerzo.

Enfoque de Sentido Común para la Gestión:

Para evitar los problemas, se sugiere un enfoque práctico dividido en cinco partes:

1. **Empezar bien:** Comprender el problema a fondo, establecer expectativas realistas y formar un equipo competente.
2. **Mantenerse:** Incentivar la retención del personal y asegurar que la calidad sea una prioridad en todas las tareas.
3. **Seguimiento del progreso:** Monitorear el avance del proyecto a través de los entregables (ej: especificaciones, código) y utilizar métricas para evaluar si se está cumpliendo la planificación.
4. **Tomar decisiones inteligentes:** Preferir soluciones sencillas y componentes de software existentes. Identificar y eliminar los riesgos a tiempo y asignar más tiempo para tareas complejas o de alto riesgo.
5. **Análisis "Post-mortem":** Al finalizar el proyecto, se debe documentar lo que se hizo bien y lo que se hizo mal, extrayendo lecciones para futuros proyectos.

El Principio "WWWWWHH" de Boehm: Barry Boehm propuso este principio, basado en siete preguntas, para crear un plan de proyecto simple y efectivo:

- **¿Por qué?:** ¿El proyecto tiene un propósito de negocio válido?
- **¿Qué? y ¿Cuándo?:** ¿Cuáles son las tareas y los hitos clave?
- **¿Quién? y ¿Dónde?:** ¿Quiénes son los responsables (dentro y fuera del equipo) de cada función?
- **¿Cómo?:** ¿Cuál será la estrategia técnica y de gestión?
- **¿Qué cantidad?:** ¿Cuántos recursos se necesitan?

Prácticas Críticas para Proyectos Exitosos:El Concilio Airlie identificó una serie de prácticas críticas que distinguen a los proyectos exitosos. Para determinar si un proyecto las está siguiendo, se pueden hacer preguntas clave:

- **Gestión de Riesgos:** ¿Cuáles son los riesgos principales y cuál es su impacto?
- **Estimación:** ¿Cómo se estimó el tamaño del software y los costos?
- **Métricas:** ¿Se usan métricas para detectar problemas de forma temprana?
- **Seguimiento de Defectos:** ¿Se registran y siguen los defectos encontrados en las revisiones y pruebas?
- **Gestión del Personal:** ¿Cuál es la tasa de rotación del personal?

Si un equipo no puede responder adecuadamente a estas preguntas, sus prácticas de proyecto deben ser revisadas y mejoradas.

Planificación de proyectos de software

Introducción Planificación de Proyectos:

- **¿Cuáles son las entradas?**
- Recursos
- Actividades
- Costos
- Riesgos
- Esfuerzo
- Limitaciones
- Complejidad

- **Preguntas que me debo hacer**
- ¿Qué herramientas debo usar?
- ¿Cuál es el nivel de precisión que busco?
- ¿Cómo se estimará la contingencia?
- ¿Cómo se va a controlar?
- ¿Cuándo y cómo se presentan los avances? (milestones/hitos)

Actividades:

- Establecer el ámbito
- Factibilidad
- Riesgos
- Definir recursos

    - Humanos
    - De SW reutilizables
    - Ambientales

- Costo y esfuerzo
    - Descomponer
    - Dimensionar

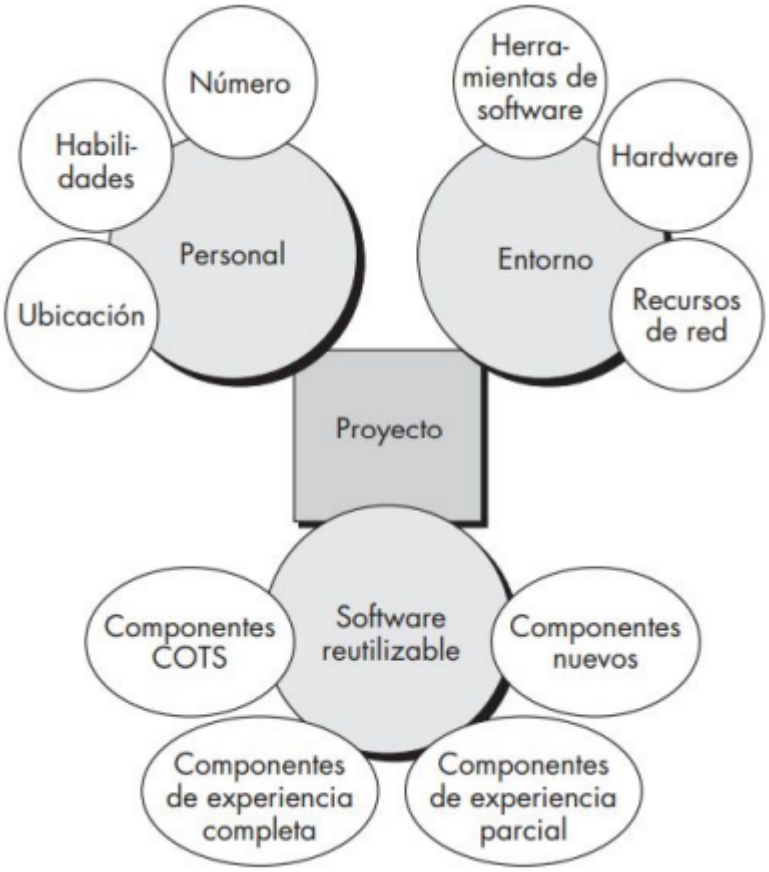
- Cronograma/Calendario
    - Tareas significativas
    - Red/Secuencia
    - Herramienta calendarización
    - Mecanismo seguimiento

Recursos

Características:

1. **Descripción:** ¿Qué es el recurso? (Ej: "Desarrollador Senior Java", "Licencia de Photoshop", "Servidor de testing").
2. **Enunciado de disponibilidad:** ¿Cuánto hay? (Ej: "2 desarrolladores", "5 licencias", "1 servidor disponible 24/7").
3. **Cuándo:** ¿En qué fecha estará disponible? (Ej: "A partir del 1 de octubre").

4. **<u>Duración:</u>** ¿Por cuánto tiempo? (Ej: "Por 3 meses", "Para toda la fase de desarrollo").



El número de personas requeridas para un proyecto de software puede determinarse sólo después de hacer una estimación del esfuerzo de desarrollo (por ejemplo, persona-meses).
Desarrollo/Diseño basado en componentes pone el énfasis en la reusabilidad

<u>Recursos a utilizar:</u> Personal. Habilidades. SW reutilizable. Componentes existentes, comerciales, nuevos. Entorno de desarrollo. HW y SW para el desarrollo y producción.

<u>La precisión se basa en:</u> El grado con que se estimó el tamaño del SW La habilidad para traducir la estimación en personas, tiempo y dinero. Introducción a la administración de proyectos

<u>Planificación de Proyectos de Software:</u>La planificación es la primera actividad en la gestión de un proyecto de software y es crucial para su éxito. Aunque no es una ciencia exacta, se basa en la **estimación** de recursos, costos y tiempo para mitigar la incertidumbre. La estimación es un "arte" que requiere experiencia, datos históricos y el coraje para confiar en predicciones cuantitativas.

Observaciones sobre la Estimación:

La fiabilidad de una estimación se ve afectada por tres factores clave:

- **Complejidad del proyecto:** La dificultad del proyecto es relativa a la experiencia del equipo. Lo que es complejo para un equipo puede ser trivial para otro más experimentado.
- **Tamaño del proyecto:** A medida que el tamaño del proyecto aumenta, la interdependencia entre los elementos del software crece rápidamente, lo que dificulta la descomposición y aumenta la probabilidad de que algo salga mal.
- **Incertidumbre estructural:** Se refiere a qué tan bien están definidos los requisitos, la facilidad para subdividir funciones y la naturaleza jerárquica de la información a procesar. Un alto grado de incertidumbre estructural aumenta el riesgo de la estimación.

El riesgo en una estimación se mide por el grado de incertidumbre en las predicciones. Un planificador debe ser consciente de que los cambios en los requisitos del cliente afectan directamente a la estabilidad del costo y el cronograma. Los enfoques modernos de ingeniería de software permiten revisar y ajustar las estimaciones a medida que se obtiene más información.

Ámbito del Software:La primera actividad de planificación es definir el ámbito del software, que describe de manera clara y no ambigua el control y los datos a procesar, la función, el rendimiento, las restricciones y las interfaces del sistema.

Obtención de la información:Al inicio del proyecto, la información es a menudo "borrosa". Para obtenerla, se recomienda el uso de preguntas de contexto libre que ayuden a comprender el problema y los objetivos del cliente. Estas preguntas pueden ir desde "¿Quién está detrás de esta solicitud?" hasta "¿Hay algo más que deba preguntarle?".

Es fundamental evitar la mentalidad de "nosotros y ellos" entre el cliente y el equipo de desarrollo. Técnicas como FAST (técnicas para facilitar las especificaciones de la aplicación) fomentan el trabajo en equipo para identificar y refinar los requisitos.

Viabilidad:Una vez definido el ámbito, se debe determinar si el proyecto es **factible**. La viabilidad tiene cuatro dimensiones sólidas:

- **Tecnología:** ¿Se puede construir el software con la tecnología actual?
- **Financiación:** ¿El costo es asumible?
- **Tiempo:** ¿Se puede entregar a tiempo para ser competitivo?
- **Recursos:** ¿La organización tiene los recursos necesarios?

Un estudio de viabilidad es crucial para evitar proyectos "condenados desde el comienzo", ya que no todo lo imaginable es factible en el mundo del software.

Ejemplo de Ámbito (SCCT):El texto ilustra la definición del ámbito con un Sistema de Clasificación de Cinta Transportadora (SCCT). Este ejemplo muestra cómo las funciones (leer códigos de barras), el rendimiento (velocidad de la cinta) y las restricciones (hardware, memoria) están íntimamente relacionadas. Cambiar cualquiera de estos elementos, como aumentar la velocidad de la cinta, puede afectar drásticamente el esfuerzo de desarrollo y, por ende, el costo.

El planificador también debe evaluar las interfaces con otros elementos del sistema (hardware, software existente, usuarios) para comprender su impacto en los recursos y el cronograma. Si no existe una especificación del sistema, el planificador debe asumir el rol de analista para obtener esta información crucial antes de la estimación.

Tareas de administración de proyectos

1. Establecer el ámbito del proyecto
2. Determinar la factibilidad
3. Analizar los riesgos
4. Definir recursos requeridos
    a. Humanos
    b. Software reutilizable
    c. Ambientales
5. Estimar costo y esfuerzo.
    a. Descomponer b. Dimensionar
6. Desarrollar un calendario del proyecto
    1. Tareas significativas
    2. Red / Secuencia de tareas
    3. Herramienta de calendarización
    4. Mecanismos de seguimiento

El ámbito describe las funciones y características que se le entregarán a los usuarios finales:

1. En las próximas secciones analizaremos en detalle:

- Definir recursos requeridos
- El proceso de estimación
- El proceso de calendarización

Componentes comerciales: T

- Telerik.

Componentes de experiencia completa (riesgo bajo): -

- NHibernate.
- Entity Framework

Componentes experiencia parcial (riesgo medio):

- Seguridad proyecto anterior.

Componentes nuevos - Desde cero Ambientales:

- HW y SW
- Servidores
- prueba Impresoras 3D
- Salas de reunión

Recursos del Proyecto: La planificación de un proyecto de software comienza con la estimación de los recursos necesarios, los cuales se pueden clasificar en una pirámide:

- **Base:** El entorno de desarrollo (EIS), que incluye el hardware y el software (herramientas) que dan soporte al proyecto.
- **Nivel intermedio:** Los componentes de software reutilizables, que pueden reducir drásticamente los costos y el tiempo de entrega.
- **Cima:** El **personal**, que es el recurso primario y cuya cantidad se determina después de estimar el esfuerzo total.

Recursos Reutilizables: La ingeniería de software basada en componentes (ISBC) enfatiza la reutilización de bloques de software ya existentes. Estos componentes deben estar catalogados, estandarizados y validados para una reutilización eficiente. Se clasifican en cuatro categorías, con riesgos de uso decrecientes:

- **Componentes ya desarrollados (CCYD):** Comprados a terceros o de proyectos anteriores, listos para usar y con riesgo bajo.
- **Componentes ya experimentados:** De proyectos similares anteriores; el equipo ya tiene experiencia, por lo que el riesgo de modificación es bajo.
- **Componentes con experiencia parcial:** Requieren una modificación sustancial y el equipo tiene experiencia limitada, por lo que el riesgo es alto.
- **Componentes nuevos:** Creados desde cero para el proyecto actual.

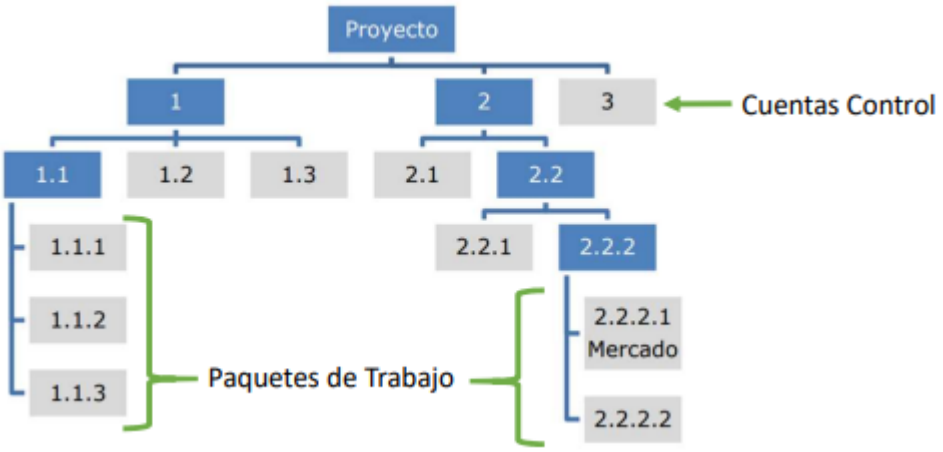
Estimación de proyectos:Demasiadas variables (humanas, técnicas, ambientales, políticas) -> afectan el costo final.

Descomposición:

- ETD/WBS (Estructura de desglose de trabajo): Organigrama jerárquico del proyecto donde se subdivide el mismo en menores componentes.
- El de más bajo nivel se llama “Paquete de trabajo”.
- Consiste en dividir en componentes menores para facilitar la planificación y control.
- Es un organigrama jerárquico donde se va subdividiendo en menores componentes.
- El nivel más bajo se denomina paquete de trabajo, generalmente se agrupan bajo “Paquetes de control”.
- No define secuencia.

Introducción a la administración de proyectos

1. Pantallas: Interfaz de usuario
2. Reportes
3. Componente 3GL: Componentes que probablemente se requieran para construir la aplicación.



Descomposición

- Definir cada actividad
- Identificar:
  - Nombre
  - Código
  - Descripción
  - Predecesora
  - Relación de dependencia
  - Restricciones

- etc

Identificador en la EDT	3.4.1
Nombre	Estudio de mercado
Código	3.4.1.2
Descripción	Estimar ingresos de turistas al país
Actividad predecesora	3.4.1.1
Actividad sucesora	3.4.1.3
Relación de dependencia	discrecional
Adelanto o retraso	Comenzar cuando 3.4.1.1 tiene 50% de avance
Requisito de recursos	1 licenciado en comercialización y 1 notebook
Fechas impuestas	No finalizar después del 15-08-2011
Restricciones	Presupuesto máximo de 50 horas
Supuestos	Se accede a base de datos del gobierno
Persona responsable	Juana Pingo
Lugar de realización	Oficina de +C
Nivel de esfuerzo	Discreto / prorrateado

Técnicas de Descomposición; La estimación basada en la descomposición se enfoca en "dividir y vencer" para simplificar el problema. Se descompone el problema o el proceso, y se estima cada parte individualmente.

Estimación basada en el problema: Es una técnica para predecir el esfuerzo, costo y tiempo de un proyecto de software. Se "dimensiona" (calcula el tamaño) del software para luego aplicar métricas de productividad de proyectos pasados. Los dos métodos principales son:

1. Estimación por LDC (Líneas de Código) es directa

- **¿En qué se enfoca?** En descomponer el software en funciones (ej: "interfaz de usuario", "gestión de base de datos").
- **¿Cómo funciona?**
    1. Se desglosa el proyecto en funciones y subfunciones.
    2. Para cada una, se estima un rango de LDC: optimista, más probable y pesimista.
    3. Se calcula un valor esperado (VE) usando una fórmula ponderada:
       `VE = (Optimista + 4 * Más Probable + Pesimista) / 6`
    4. Se suman los valores esperados de todas las funciones para obtener el total de LDC del proyecto.
    5. Este total se multiplica por una métrica de productividad histórica (ej: LDC por persona-mes) para obtener el esfuerzo y costo final.

Característica clave: Requiere una descomposición muy detallada del software en funciones.

2. Estimación por PF (Puntos de Función) es indirecta

- ¿En qué se enfoca? En los componentes de información del sistema, no en sus funciones. Se cuantifican:
    1. Entradas, Salidas, Consultas, Archivos lógicos internos e Interfaces externas.
- ¿Cómo funciona?
    1. Se identifican y cuantifican los componentes de información anteriores.
    2. A cada uno se le asigna un peso (bajo, medio, alto) según su complejidad.
    3. Se suma el total de Puntos de Función sin ajustar.
    4. Se calcula un Factor de Ajuste de Complejidad basado en 14 características del sistema (ej: rendimiento crítico, entrada de datos en línea, etc.).
    5. Se aplica la fórmula para obtener el total de PF ajustados: `PF_Ajustados = PF_Sin_Ajustar * [0.65 + (0.01 * Suma_de_Factores_de_Ajuste)]`
    6. Este total se multiplica por una métrica de productividad histórica (ej: PF por persona-mes) para obtener el esfuerzo y costo.

Característica clave: Se centra en qué hace el sistema desde la perspectiva del usuario, no en cómo se construye.

Puntos Clave en Común y Consejos

- Ambos métodos usan datos históricos de productividad de proyectos anteriores para convertir el "tamaño" (LDC o PF) en esfuerzo y costo.
- Es crucial agrupar los proyectos históricos por similitud (dominio, complejidad, tamaño del equipo) para usar la métrica de productividad correcta. Nunca uses un promedio general.
- Ningún método es infalible. Siempre se debe verificar la estimación con otra técnica y aplicar el sentido común y la experiencia.

Resumen Final

- Usa LDC si puedes descomponer el software en funciones técnicas detalladas y tienes datos históricos de productividad basados en código.

- Usa PF si prefieres medir el tamaño con base en la funcionalidad que ve el usuario, lo que es útil desde etapas tempranas del proyecto, incluso antes de definir cómo se programará.

Estimación basada en proceso: Este método estima el costo y esfuerzo de un proyecto desglosando el trabajo necesario para realizar cada actividad del proceso de desarrollo (análisis, diseño, codificación, pruebas, etc.), en lugar de enfocarse en el tamaño del producto final (como en LDC o PF).

¿Cómo funciona?

1. Desglose del Trabajo: Se identifican todas las funciones del software (ej: "Interfaz de usuario", "Base de datos") y todas las actividades/tareas del proceso que se aplicarán a cada una (ej: "Análisis de requisitos", "Diseño detallado", "Codificación", "Pruebas de integración").
2. Matriz de Estimación: Se crea una tabla o matriz donde:
    - Las filas son las funciones del software.
    - Las columnas son las actividades del proceso.
    - Cada celda representa el esfuerzo estimado (en personas-mes, personas-hora) para aplicar esa actividad a esa función.
3. Cálculo de Costos: Se suma el esfuerzo de todas las celdas para obtener el esfuerzo total del proyecto. Luego, este esfuerzo se multiplica por los costos laborales (que pueden variar según la seniority del personal asignado a cada tarea) para obtener el costo total estimado.

Ejemplo Práctico

Para el proyecto de software CAD, se creó una matriz donde para cada función (Interfaz de usuario, Análisis geométrico, etc.) se estimó el esfuerzo para tareas como:

- Comunicación con el cliente
- Planificación
- Análisis de riesgos
- Análisis de requisitos
- Diseño
- Codificación
- Pruebas

Resultado del ejemplo:

- Esfuerzo total estimado: 46 personas-mes
- Costo total estimado: £230,000 (asumiendo una tarifa plana de £5,000 por persona-mes)
- Hallazgo clave: El 53% del esfuerzo se destinó a las actividades "frontales" (análisis y diseño), destacando su importancia.+

Comparación y Consejo Clave

- Ventaja: Es muy detallada y permite ver exactamente en qué se gastará el tiempo y el dinero.
- Mejor Práctica: Nunca confíes en una sola técnica de estimación. Este método debe usarse para contrastar los resultados obtenidos con las estimaciones basadas en el problema (LDC y PF).
- ¿Qué hacer si las estimaciones no coinciden? Si los resultados de los distintos métodos (Proceso, LDC, PF) varían mucho (ej: 46 vs. 58 personas-mes), es una señal de alarma. Las causas usuales son:
    1. Mala interpretación del alcance: No se entendió bien lo que el proyecto debe entregar.
    2. Datos históricos erróneos: Las métricas de productividad (LDC/pm o PF/pm) usadas no son apropiadas para este proyecto en particular.

Conclusión: Una estimación fiable surge cuando los diferentes métodos independientes arrojan resultados similares. Si no es así, hay que investigar y corregir la causa de la discrepancia antes de proseguir.

Estimación con Casos de uso: Este método utiliza los Casos de Uso (historias que describen la interacción entre un usuario y el sistema para lograr un objetivo) como unidad principal para medir el tamaño y la complejidad del software, y así estimar el esfuerzo requerido.

¿Cómo funciona?

El proceso general sigue estos pasos:

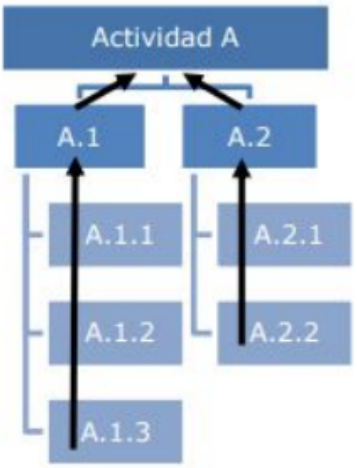
1. **Identificar y Ponderar los Casos de Uso:** No todos los casos de uso son igual de complejos. Se clasifican según su cantidad de transacciones o su complejidad técnica en simples, medios o complejos.
    - **Simple:** Tiene de 1 a 3 pasos o transacciones (ej: "Login básico").
    - **Medio:** Tiene de 4 a 7 transacciones (ej: "Realizar una compra" que incluye seleccionar producto, ingresar datos de envío y pago).
    - **Complejo:** Tiene más de 7 transacciones y puede involucrar lógica de negocio complicada (ej: "Calcular una declaración de impuestos").

2. **Asignar Puntos o Horas:** A cada categoría de caso de uso se le asigna un valor predeterminado, ya sea en Puntos de Caso de Uso (una medida abstracta de tamaño) o directamente en horas de trabajo.
    - Ejemplo de valores típicos:
        - **Caso de Uso Simple:** 5 puntos o 15-20 horas de trabajo.
        - **Caso de Uso Medio:** 10 puntos o 30-40 horas.
        - **Caso de Uso Complejo:** 15 puntos o 60-80 horas.
3. **Calcular el Esfuerzo Total:**
    - Se multiplica la cantidad de casos de uso de cada tipo por su valor en puntos/horas.
    - Se suman todos los resultados para obtener el tamaño total del proyecto en puntos u horas.
    - Esfuerzo Total = (Nº de UC Simples * Valor Simple) + (Nº de UC Medios * Valor Medio) + (Nº de UC Complejos * Valor Complejo)
4. **Aplicar la Productividad del Equipo:** Se convierte el tamaño total en esfuerzo (personas-hora o personas-mes) usando la velocidad o productividad histórica del equipo.
    - **Ejemplo:** Si el equipo tiene una productividad de 20 puntos por persona-mes, y el proyecto tiene 200 puntos, el esfuerzo estimado será de 10 personas-mes.

Ventajas y Desventajas

- **Ventaja:** Es muy útil en las etapas iniciales del proyecto (cuando los requisitos se entienden en términos de funcionalidad para el usuario) y es fácil de comunicar a clientes y partes interesadas no técnicas.
- **Desventaja:** La estimación puede ser imprecisa si los casos de uso no están bien definidos o si los factores de complejidad no se evalúan correctamente. No considera la complejidad técnica interna (algoritmos, integraciones) tan bien como otros métodos (como LDC).

Conclusión: La Estimación con Casos de Uso es una técnica ágil y orientada al usuario que complementa a las basadas en problema (LDC/PF) y proceso. Es ideal para proyectos donde los requisitos se capturan como Casos de Uso, pero, como con todas las técnicas, debe ser contrastada con otros métodos y ajustada según la experiencia e historial del equipo para ser más precisa.



Otros modelos de estimación

- COCOMO
- COCOMO II
- COnstructive COst MOdel
- Para realizar la estimación utiliza “Puntos de objetos”

$PROD = NOP\ PERSONA - MES$

$E = NOP\ PRO$

Tipo de objeto	Peso de complejidad		
Pantalla	1	2	3
Reporte	2	5	a
Componente 3GL			10

1. Pantallas: Interfaz de usuario
2. Reportes
3. Componente 3GL: Componentes que problablemente se requieran para construir la aplicación. vvvv

Estimación

- Introducción a la administración de proyectos
- Juicio Experto (es la voz de la experiencia)
- Estimación Análoga (proyectos anteriores)
- Análisis de Reserva (backup)
- Paramétrica
- PERT - 3 Valores

Juicio Experto:

Análisis de Reserva: se agrega una reserva de tiempo por contingencias.

Estimación Análoga: Juicio Experto + Proyectos anteriores.

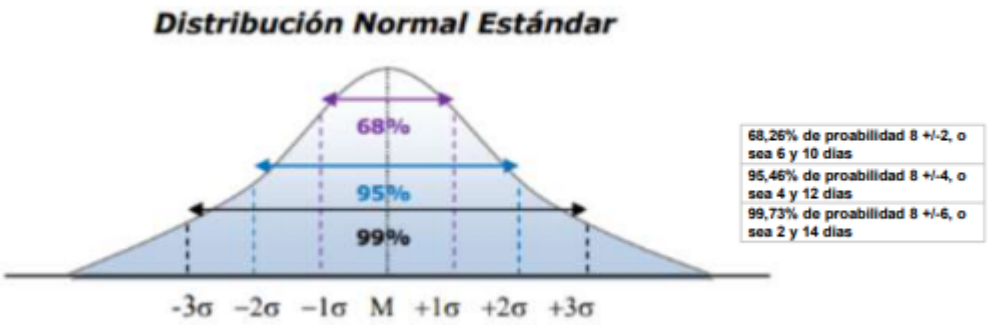
Paramétricas: Depende de variables. Históricos, Proyectos previos, Estándares Técnicos grupales para la toma de decisiones.

Estimación por tres valores: Optimista, Pesimista, Más probable. PERT Media.

Fomularios	Horas
5	18
2	7
15	53
10	47
1	3
15	50
14	40
2	6
4	10
11	30
5	16
14	40
16	59
9	30
5	14
11	35
9	25
16	48
14	43

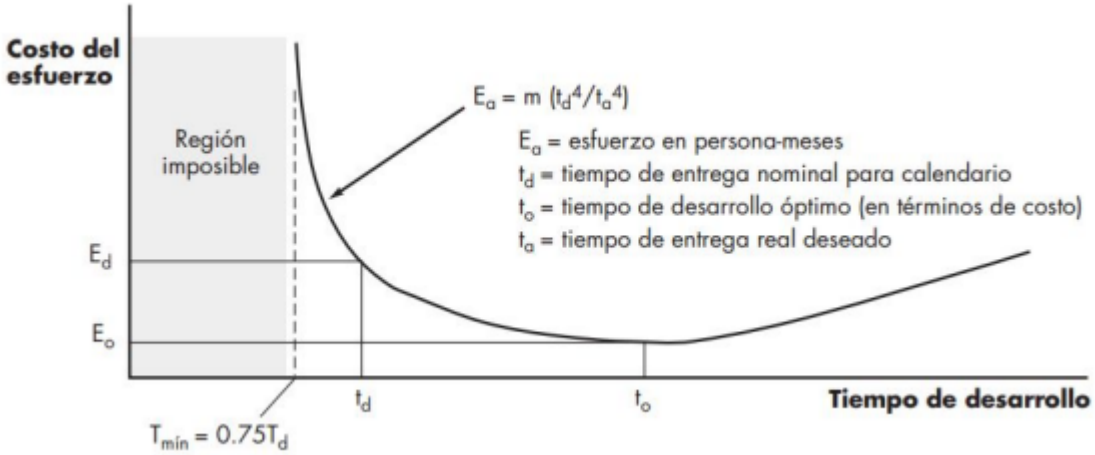
Utiliza el historial de proyectos Utilizar parámetros en base a información histórica para poder estimar la duración de una actividad futura.
El valor R2 de 0,92 indica que la variable X (número de formularios) está explicando en un 92% a la variable Y (duración en horas). Valores de R2 superiores a 0,7 indican que los datos son aceptables para proyectar el futuro.
Reemplazando la incógnita de 10 formularios (X = 10) en la ecuación, se obtiene lo siguiente: Y = 3,2699 x 10 – 0,4229 = 32,28 y tiene un 92% de confianza.

Estimación – 3 valores (Pert):Tarea que tiene una duracion optimista de 4, mas probable 7 y pesimista de 16Duracion est (4 + 4*7+16)/6 =8 Desv estandard (16 - 4)/6 = 2



Cronograma

Personal y esfuerzo



Compresión del cronograma: Consiste en acortar el cronograma sin modificar el alcance

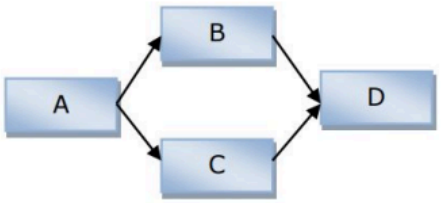
Fast Tracking y Crashing

Técnica	Mecanismo (¿Cómo se acorta el tiempo?)	Impacto Principal
Fast Tracking (Ejecución Rápida)	Consiste en **realizar tareas en paralelo** que originalmente estaban planificadas para ejecutarse en secuencia. Se enfoca en la **paralelización de tareas** del camino crítico.	Aumenta los riesgos.
Crashing (Compresión)	Consiste en **agregar recursos** (ajustar o incrementar los recursos) a las tareas del camino crítico. El objetivo es acortar la duración de las tareas **tratando de gastar lo menos posible**.	Aumenta el costo del proyecto, aunque se busca minimizarlos.

En resumen, Fast Tracking implica la paralelización de tareas del camino crítico, lo que incrementa el riesgo. Por su parte, Crashing implica ajustar o incrementar los recursos para reducir el tiempo, manteniendo el alcance original.

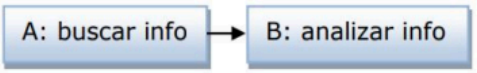
Secuenciar Actividades

Diagramación por precedencia (PDM)

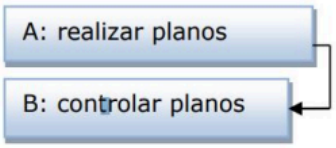


Tipos de Precedencia

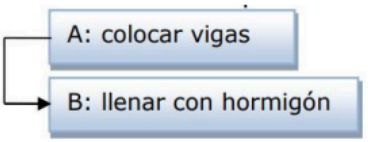
Final a Inicio



Final a Final



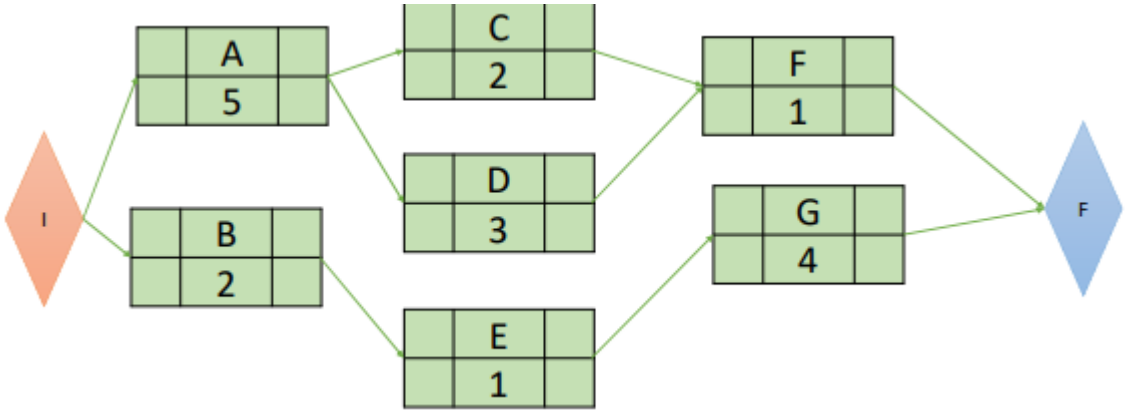
Inicio a Inicio



Tipo de dependencias:

- **Dependencias obligatorias:** también conocidas como "Lógica Dura" Una dependencia obligatoria es inherente a la naturaleza del trabajo (ejemplo: se debe diseñar antes de construir) o es requerida por el contrato.
- **Dependencias discrecionales:** también conocidas como "Preferida, Preferencial o Lógica Blanda". Existen otras formas en que se podría realizar el trabajo, pero este es el enfoque preferido. Puedes cambiar una dependencia discrecional si es necesario, mientras que no puedes cambiar fácilmente los otros tipos de dependencias.
- **Dependencia externa:** Esta dependencia se basa en las necesidades o deseos de una parte externa del proyecto (ejemplo: gobierno o proveedores).
- **Dependencia interna:** Esta dependencia se basa en las necesidades del proyecto y puede ser algo que el equipo del proyecto puede controlar.

Cálculo del camino crítico



ES (Inicio más Temprano)
EF (Finalización más Temprana)
LS (Inicio más Tardío)
LF (Finalización más Tardía)

Se posiciona así:

ES	Activi dades	EF
LS	Duracion	LF

Reglas:

Regla	Nombre	Fórmula	Se usa para
Máximo	ES es de una actividad	Es sucesora = MAXIMO EF predecesoras	Pase Hacia Adelante
Mínimo LF de una actividad	LF de una actividad	LF predecesora = MINIMO LS sucesoras	Pase Hacia Atrás
Duración del proyecto	LF final		LF Última = MÁXIMO EF final (Se usa este valor para empezar el Pase Hacia Atrás)
Paso Final	El último paso para responder completamente al problema es encontrar el **Camino Crítico**:	Holgura (H) H=LS−ES o H=LF−EF	

Para empezar se asume que el ES(inicio más temprano) empieza en 0

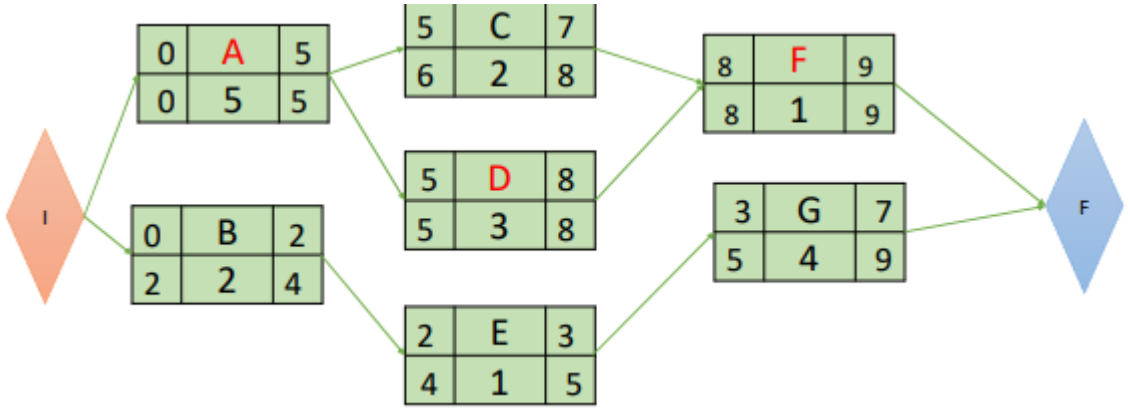
Toda actividad que forma parte del Camino Crítico tiene una Holgura (H) igual a cero.

Fórmulas:

EF = ES + Duración
LS = LF - Duración

La holgura es = LF - EF

El rombo I es de Inicio y el rombo de F de Fin



Camino crítico: Conjunto de actividades que tienen una holgura de 0 (podría tener más de uno)
A-D-F

Herramientas de calendarización y seguimiento

Tareas básicas

1. Definir el calendario
2. Establecer la fecha de inicio del proyecto
3. Incorporación de tareas
4. Relaciones entre tareas / dependencias
5. Definición de hitos
6. Agrupar tareas
7. Definición de recursos
8. Cronograma (Diagrama de Gantt)
9. Camino critico
10. Línea base

Resumen

1. Establecer el ámbito del proyecto
2. Determinar la factibilidad
3. Analizar los riesgos
4. Definir recursos requeridos
5. Estimar costo y esfuerzo
6. Definir actividades
7. Secuenciar las actividades
8. Desarrollar un calendario

Modelos Empíricos de Estimación:Los modelos empíricos de estimación usan **fórmulas matemáticas** para predecir el esfuerzo (en personas-mes) a partir del tamaño del software (LDC o PF). Estos modelos, como **COCOMO**, se basan en datos históricos de proyectos anteriores y tienen un componente de ajuste que considera factores como la complejidad y la experiencia del equipo. Es importante **calibrar** estos modelos con datos locales para obtener resultados precisos.

El Modelo COCOMO II:**COCOMO II** es una jerarquía de modelos de estimación que se adapta a diferentes etapas del proyecto:

- **Composición de Aplicación:** Para las primeras etapas, utiliza **Puntos Objeto (PO)**, una medida indirecta que cuenta pantallas, informes y componentes. El esfuerzo se estima a partir del número de PO ajustado por factores como la reutilización y la productividad del equipo.
- **Diseño Previo y Posterior a la Arquitectura:** Modelos más avanzados que usan Puntos de Función y Líneas de Código.

La Ecuación del Software:Este es un modelo dinámico que utiliza el tamaño del software (LDC) y la duración del proyecto para estimar el esfuerzo. Incluye un **parámetro de productividad (P)** que refleja la madurez del proceso y la experiencia del equipo.

La Decisión "Desarrollar o Comprar":Los gestores a menudo enfrentan la decisión de **adquirir software ya hecho o desarrollarlo internamente**. Las opciones de adquisición incluyen comprar un producto completo, reutilizar componentes o contratar a una empresa externa (**outsourcing**).

Evaluación de las Opciones

Para tomar una decisión informada, se deben seguir estos pasos:

1. **Definir las especificaciones** del software.
2. **Estimar el costo interno** y la fecha de entrega.

3. **Identificar y evaluar** posibles productos o componentes candidatos.
4. **Comparar** las opciones en una matriz, incluyendo factores como la calidad, el soporte del vendedor y la reputación.
5. **Analizar las condiciones** de costo, tiempo y soporte para cada opción.

Análisis de Árbol de Decisiones: Una herramienta útil es el **árbol de decisiones**, que permite calcular el **costo esperado** para cada opción (desarrollar, reutilizar, comprar, contratar) al multiplicar el costo estimado por la probabilidad de un resultado específico. Sin embargo, no se debe basar la decisión únicamente en el costo; también se deben considerar otros criterios como la disponibilidad y la conformidad con los requisitos.

Outsourcing: El outsourcing es la práctica de contratar a terceros para que realicen las actividades de ingeniería de software a un costo menor. Puede ser una decisión estratégica o táctica.

- **Ventajas:** Ahorro de costos al reducir personal e infraestructura.
- **Desventajas:** Pérdida de control sobre el software, lo que puede ser riesgoso si el software es una tecnología clave para la empresa.

Las herramientas automáticas de estimación son software que implementa técnicas de descomposición y modelos empíricos para ayudar a los gestores a estimar el costo y el esfuerzo de un proyecto de software. Sirven como un punto de partida para la estimación, no como una fuente única de verdad, ya que los resultados pueden variar y diferir de los valores reales.

Funciones Clave de las Herramientas: Estas herramientas automatizadas realizan seis funciones genéricas para la planificación:

1. **Dimensionamiento de las entregas:** Estiman el "tamaño" del software en términos de líneas de código (LDC), puntos de función (PF), o la cantidad de pantallas y documentos.
2. **Selección de actividades:** Permiten al usuario elegir el marco de proceso adecuado y las tareas de ingeniería de software para el proyecto.
3. **Predicción del personal:** Ayudan a determinar el número de personas necesarias, teniendo en cuenta que la relación entre la cantidad de personal y el esfuerzo no es lineal.
4. **Predicción del esfuerzo:** Utilizan modelos de estimación para calcular el esfuerzo total requerido, relacionando el tamaño del producto con el trabajo necesario para crearlo.
5. **Predicción del costo:** Calculan los costos del proyecto asignando tarifas laborales a las actividades estimadas en el paso anterior.

6. **Predicción del cronograma:** Generan un borrador del cronograma del proyecto, distribuyendo el esfuerzo en el tiempo basándose en modelos de distribución recomendados.

Resumen de la Planificación de Proyectos

En resumen, la planificación de un proyecto de software busca estimar la duración, el esfuerzo y el personal necesarios, además de predecir los recursos (hardware y software) y los riesgos.

El proceso de estimación se apoya en tres técnicas principales:

- **Descomposición:** Se desglosan las funciones del software en partes más pequeñas y se estima el tamaño (LDC, PF) o el esfuerzo de cada una.
- **Modelos empíricos:** Se utilizan fórmulas matemáticas basadas en datos históricos para predecir el esfuerzo y el tiempo.
- **Herramientas automáticas:** Programas de software que aplican los modelos empíricos y las técnicas de descomposición.

Para obtener una estimación precisa, es recomendable utilizar al menos dos de estas técnicas y comparar y conciliar los resultados. Aunque la estimación nunca será una ciencia exacta, el uso de buenas técnicas sistemáticas y datos históricos confiables puede mejorar significativamente su precisión.

Scrum

1. El Objetivo de Scrum: Scrum es un marco de trabajo (framework) diseñado para describir, simplificar y mejorar la manera en que se realizan los proyectos, especialmente en entornos donde el trabajo no es mecánico ni predecible, como el desarrollo de productos o conocimiento.

- **Problema que aborda:** Mucha gente cree que está usando Scrum, pero en realidad sigue aplicando el "mismo modelo de siempre" (el paradigma industrial) con otra etiqueta, lo que lleva al fracaso o a la incomprensión de su verdadero potencial.
- **La Clave:** Scrum representa un nuevo paradigma o modelo mental para entender y organizar el trabajo, y cambiar modelos mentales es incómodo y difícil.

2. La Importancia de la Visión (El Paradigma Ágil): La anécdota de Hans y Frida ilustra un principio fundamental de Scrum (y del enfoque Ágil): la necesidad de tener una Visión Clara y de ser adaptable.

- **El Error de Hans (Primer intento):** Tratar de planificar el camino paso a paso en un entorno incierto (el arroyo oscuro y con niebla). Solo veía el siguiente paso, pero no su destino final.
- **El Acierto de Frida (Segundo intento):** Le dice a Hans que, antes de cada salto, debe levantar la vista y elegir la opción que más le acerque a la meta (la vela en la ventana).

3. El Conflicto de Paradigmas: Industrial vs. Conocimiento

Paradigma Industrial (Obsoleto)	Paradigma del Conocimiento (Necesario)
Enfoque: Órdenes, control, procedimientos rígidos, predictibilidad.	**Enfoque:** Innovación, colaboración, capacidad de establecer relaciones y crear estrategia.
Persona: Un "recurso" o "engranaje" que se reemplaza fácilmente.	**Persona:** Valioso por lo que sabe y su capacidad de innovar. El conocimiento es suyo y no está en un manual.
Trabajo: Mecánico y rutinario (ej: martillar clavos). El conocimiento es explícito (está en el procedimiento).	**Trabajo:** No es mecánico ni predecible. Es desarrollar, crear, diseñar. El conocimiento es tácito y emerge del equipo.

4. Tipos de Proyectos: sirve para clasificar un proyecto según su Complejidad Relativa, es decir, qué tan difícil es *para nuestro equipo* en este momento.

Tipo de Proyecto	Requerimientos	Herramientas	Enfoque de Trabajo Óptimo
Simple	Conocidos (Sabemos qué hacer).	Conocidas (Sabemos cómo hacerlo).	Línea de Producción / Cascada. Planificación detallada, resultados previsibles. Poca innovación.
Caótico	Desconocidos (No sabemos qué necesitamos).	Desconocidas (No sabemos cómo empezar).	Prueba y Error. Se vive en la investigación pura sin un camino claro.
Complejo	Inciertos (Hay dudas o posibles cambios).	Inciertas (Son novedosas o difíciles de usar).	Scrum. Proyectos riesgosos y creativos, comunes en el trabajo del conocimiento.

Conclusión: Scrum está diseñado para proyectos en la zona de complejidad, donde hay incertidumbre tanto en el objetivo (Requerimientos) como en el método (Herramientas).

5. Surfear el Borde del Caos (Límites y Libertad): La clave para gestionar un proyecto Complejo es alcanzar un "equilibrio inestable" que permita ser creativos y a la vez productivos.

- **El Problema:** La infinidad de posibilidades (el Caos) paraliza la acción.
- La Solución (Límites): Para promover la creatividad, necesitamos establecer la cantidad mínima de límites necesarios (ejemplo: dibujar solo círculos en un minuto).
- **La Filosofía de Scrum:** Scrum actúa como un canal que balancea límites con libertad. Los límites permiten encauzar la energía y ser productivos, mientras que la libertad interna del equipo permite la innovación, el ensayo y el error.
- **Demasiados Límites:** Convierten el proyecto en Simple y aburrido, matando la innovación.

6. Complejidad Esencial vs. Accidental: Aristóteles divide la dificultad de un problema en dos tipos, lo que nos ayuda a entender para qué *no* sirve Scrum y para qué *sí* sirve.

A. Complejidad Esencial (El Corazón del Problema): Es la dificultad inherente al problema; es irrompible e imposible de simplificar sin cambiar el problema en sí (como hizo el alpinista Maestri al usar un taladro).

- **Relación con Scrum:** Scrum NO resuelve esta complejidad. Si un problema es intrínsecamente complejo (ej: predecir un proyecto de dos años con personas desconocidas), Scrum no lo hará simple.

B. Complejidad Accidental (La Grasa): Es la dificultad añadida que no es propia del problema, sino que viene "de regalo" (falta de confianza, sillas incómodas, burocracia, etc.). Es el "desperdicio" del que habla la filosofía Lean.

- **Relación con Scrum:** Scrum sí sirve para esto. Es una "pequeña maquinita" que:
    1. Detecta y hace emerger las disfuncionalidades y la "grasa" organizacional.
    2. Fuerza a la organización a enfrentarlas cara a cara.

Conclusión Final: Scrum no resuelve el problema, sino que revela los problemas de la organización que impiden resolver el problema. La decisión de resolver esas disfuncionalidades es de la empresa, no de Scrum.

Beneficios de Scrum

La Necesidad del Ciclo Adaptativo: Scrum resuelve este problema al obligarte a trabajar con el Ciclo de Vida Iterativo e Incremental. Este cambio de modelo de vida genera los siguientes cuatro beneficios esenciales:

1. Minimización del Riesgo y Maximización del Valor (Incremental)

- **Problema Lineal:** Si el proyecto falla (se acaba el tiempo o el dinero), el valor entregado es cero (ej: solo tienes tres arcos sin pelota).
- **Beneficio Scrum:** Al entregar el producto en versiones funcionales (incrementos) desde el principio (la pelota pequeña o el brote), garantizas que siempre se entrega valor al cliente, incluso si el proyecto debe detenerse antes de tiempo.

2. Aprendizaje y Entendimiento Constante (Iterativo)
    - **Problema Lineal:** Se asume que el plan original es perfecto y que el producto fue entendido a la perfección antes de empezar.
    - **Beneficio Scrum:** Fomenta el "Project Steering" (maniobrar), no el "Project Tracking" (seguir el plan). En cada ciclo corto (iteración), se obtiene Feedback Temprano del cliente. Este aprendizaje constante asegura que estás construyendo lo que realmente se necesita (ej: te das cuenta que la pelota debe ser rugosa) y no lo que creías que se necesitaba.
3. Adaptación a los Cambios (Iterativo e Incremental)
    - **Problema Lineal:** Es muy costoso y difícil cambiar el plan o el producto a mitad de camino. Los cambios en el contexto (el vecino, el alcalde) arruinan el proyecto.
    - **Beneficio Scrum:** El producto se construye de forma maleable y orgánica (incremental). Esto permite adaptar plan y producto a cualquier cambio, ya sea porque entendiste mejor lo que quería el cliente o por un cambio en el contexto externo.

Resumen Final: Scrum es el conjunto de reglas que te permite trabajar en un Ciclo de Vida Iterativo e Incremental. Este ciclo de vida es la única forma efectiva de gestionar proyectos complejos porque te permite aprender y adaptarte continuamente para garantizar la entrega de valor a pesar de la incertidumbre.

Reglas del Juego: Scrum Roles y Artefactos

1. Definición de Scrum
    - Scrum NO es una Metodología ni un Proceso rígido.
    - Scrum es un Framework o Marco de Trabajo (Meta-Proceso).

    Propósito: Es el andamiaje que ayuda al equipo a construir, iteración a iteración, su propio proceso ideal y el producto, de forma orgánica y adaptable.

La dinámica central de Scrum consiste en pequeños saltos (iteraciones) que siempre incluyen: Planificar → Ejecutar → Inspeccionar (el producto y el proceso) → Adaptar.

2. Los 3 Roles de Scrum (Quién)

Los roles dividen la responsabilidad entre la Estrategia (el QUÉ) y la Táctica (el CÓMO), promoviendo la Auto-Gestión del equipo.

Rol	Perspectiva / Responsabilidad Principal	Enfoque
Product Owner (PO)	Maximizar el Retorno de Inversión (ROI). Define el QUÉ del producto y es el único embajador de los Stakeholders (interesados).	Estrategia (La Visión/Qué características).
Equipo de Desarrollo	Construir el producto. Poseen todos los skills necesarios (multidisciplinario). Son auto-gestionados y responsables del CÓMO.	Táctica (El Cómo desarrollar el producto).
Scrum Master (SM)	Optimizar el Proceso de Trabajo. Actúa como espejo para PO y Equipo, asegurando que sigan las reglas de Scrum.	Proceso (El Cómo mejorar el trabajo).
Funciones del SM:	Facilitador (ayuda a tomar decisiones neutrales), Coach (evoca la excelencia) y Mentor (enseña, usando la pregunta como herramienta principal).	

3. Los 4 Artefactos de Scrum: son las herramientas de trabajo que plasman la estrategia y la táctica.

Artefacto	Descripción y Responsable	Importancia Clave
Visión	La descripción somera del norte o meta final del proyecto.	Faro que guía la estrategia. Responsable: Product Owner.
Product Backlog	Lista ordenada por prioridad (usualmente maximizando la relación costo/beneficio) de todas las características (PBIs) que le faltan al producto.	Es la Estrategia del proyecto. Responsable: Product Owner.
Sprint Backlog	Las tareas que el equipo define qué debe hacer para cumplir con el subconjunto de PBIs que se comprometió a entregar en el Sprint (la iteración).	Es la Táctica del Sprint. Responsable: Equipo de Desarrollo.
Incremento Orgánico	Es la nueva versión funcional del producto al	El Valor entregado. Es la única forma de obtener feedback útil y de garantizar que

	final de cada Sprint. Debe cumplir con el Criterio de Hecho (Definition of Done).	no queda nada "a medias."

El Espíritu de Scrum: Filosofía, Empirismo y Valores

1. El Espíritu de Scrum (Reglas + Espíritu + Buenas Prácticas)

- **Scrum es Simple pero Difícil:** El *framework* (las reglas) es simple y a propósito incompleto (como una ley). Para aplicarlo correctamente, requiere interpretación (como un juez).
- **El Espíritu:** Es la intención detrás de las reglas, los valores y los principios que guían la interpretación. Sin este espíritu, Scrum se convierte en un conjunto de reuniones sin sentido ("Scrum muerto").

2. Fundamento Filosófico: Complejidad y Empirismo: Scrum es necesario porque el Trabajo del Conocimiento (como el desarrollo de *software*) es inherentemente Complejo debido a dos factores:

   Maleabilidad: El producto es fácil de cambiar, lo que hace que el cambio sea la única constante.
   Creatividad: Hay que diseñar lo inexistente (lo nuevo es lo que aporta valor).

- **Respuesta de Scrum:** Ante la complejidad, Scrum abandona el Racionalismo (creer que se puede planificar todo por adelantado) y adopta el Empirismo como filosofía de trabajo.
- **Empirismo:** Generación de conocimiento mediante la experiencia; el método es Prueba y Error.

3. El Error como Inversión: El empirismo exige cambiar la forma de ver los fallos:

- **Nueva Visión:** El error se considera una inversión, no una pérdida.
- **Razón:** Al equivocarse, se reduce el universo de posibles soluciones y se aprende qué funciona (como en los ensayos de teatro). Esto permite avanzar hacia la solución, aunque duela al orgullo.
- **Regla de Oro:** Para que el error sea una inversión, debe tener un Bajo Costo. Esto se logra con Ciclos de Feedback Cortos (Sprints cortos, Daily Meetings de 15 minutos), asegurando que "si me equivoco, sea lo antes posible."

4. Valores Finales

- **Crecimiento Orgánico (Maleabilidad):** El producto y el proceso deben ser maleables (adaptables) para absorber cambios frecuentes a bajo costo. El desarrollo incremental asegura que puedes detenerte en cualquier

momento y conservar todo el valor acumulado ("Nada se ha roto, el árbol no se ha secado").

- **Priorización (Ley de Pareto):** Dado que las estimaciones fallan, se aplica el principio de Pareto: el 20% de la funcionalidad entrega el 80% del valor. El Product Backlog se prioriza para construir ese 20% primero, maximizando el ROI.
- **La Premisa Final:** En Scrum, se cree que "Lo Perfecto es Enemigo de lo Bueno". El éxito no es completar el plan, sino maximizar el retorno de inversión de los *stakeholders* en un entorno de cambio constante.

En síntesis: El Espíritu de Scrum es un abrazo consciente a la Complejidad, eligiendo el Empirismo para aprender rápido, falla barato y priorizar el valor, en lugar de perseguir una perfección inalcanzable.